Measuring the Overhead of Trust Mechanisms in the X-Road Data Exchange Layer

Ruslan Nechshadimov
University of Tartu
Supervisor: Artjom Lind

Draft v0.2, Oktober 14, 2026

Abstract

WIP

1 Introduction

When one organisation acts on data received from another, it depends on something it did not produce and cannot check by itself. Trust is the name for that dependence. In organisational research it is defined as the willingness of a party to be vulnerable to the actions of another party, based on the expectation that the other will perform an action important to the trustor, without being able to monitor or control it [17]. In computing the same idea is phrased as a belief in the competence of an entity to act dependably, securely and reliably within a specified context [18]. In security engineering a *trusted* component is distinguished from a *trustworthy* one: the first is a component whose failure can break the security policy, the second one that will not fail [19]. A well-designed system therefore tries to need as little trust as possible, and to place the rest in a few components that are named explicitly and protected accordingly.

X-Road, the data exchange layer, is built on this principle. Its member organisations do not have to trust each other's systems directly. Their trust rests on a root of trust, a point that is accepted as trusted axiomatically [3]. In X-Road this root is centralised: it is formed by the certification authorities that vouch for the members' keys [3], together with the Central Server, which publishes who the members are and which authorities to trust, and the time-stamping authority [11]. Every message then carries evidence that can be checked against this root. It is signed by the sender's Security Server and verified by the receiver's, written into the message logs on both sides and time-stamped, and it travels over a mutually authenticated Transport Layer Security (TLS) channel, while the status of the certificates is checked through the Online Certificate Status Protocol (OCSP) [11, 1]. Trust in the other party is replaced by verification against the root of trust, and an exchange can later be used as evidence of who sent what and when. The other side of a centralised root is that it becomes a single point of failure [3], which is why alternatives to it are being studied.

Verification is not free. Signing, verification, logging and setting up the channel take computation and bytes, some of them on every message and some in the background. X-Road has been measured before, but end to end, for questions other than the cost of these mechanisms: a study of threshold signatures reports end-to-end round trips of about 32 ms with all components on one laptop and presents them as a lower bound [4]; a capacity study measures the overhead of a Security Server in front of one registry service, where database time dominates [7]; and the paper that introduced batch time-stamping gives an analytical estimate and leaves benchmarking of the implementation to future work [1]. The Nordic Institute for Interoperability Solutions (NIIS), which develops X-Road, does not publish performance data. None of these sources splits the cost of an exchange by mechanism.

Two developments make that missing split more relevant. The first is volume: the design of batch time-stamping was calibrated against about 240 million queries per year [1], while the Estonian instance now reports 3.74 billion [10], so whatever is paid per message is multiplied by a much larger base. The second is a set of proposals that would make the trust mechanisms themselves more expensive. Post-quantum signatures are expected to enlarge certificates, signatures and handshakes [5, 6]; a Dilithium5 signature, for example, takes 4595 bytes against the 256 bytes of RSA-2048 [6]. Decentralised Public Key Infrastructure (PKI) built on verifiable credentials adds a presentation to every request [2], and threshold signatures replace a local key with a distributed protocol [4]. These works describe what will become more expensive, but without a measured baseline for the present system they cannot say by how much.

The proposals above do not change X-Road as a whole but individual mechanisms. Post-quantum algorithms affect signing, verification and the TLS handshake [5], a decentralised PKI changes how certificates are checked [2], and threshold signatures change how messages are signed [4]. The cost of each change can only be judged against the cost of the mechanism it replaces, and since X-Road keeps certificate status checking and time-stamping out of the request path by design [1], it also matters which of these costs are paid on every message. Larger post-quantum signatures and certificates make messages bigger, and the growth in volume means more messages processed at the same time, so the costs have to be known as a function of message size and load. Finally, the most recent figures were obtained with all components on one machine, which their authors present as a lower bound [4], whereas the Security Servers of two organisations normally communicate over a network, and the NIIS report expects post-quantum handshakes to add round trips on it [5]. This leads to the following research questions:

RQ1
: How much does each trust mechanism — TLS between the Security Servers, message signing, signature verification, certificate status checking, time-stamping and message logging — contribute to the latency of an exchange and to the bytes on the wire, and which of these costs is paid on every message rather than deferred?

RQ2
: How do those contributions change with message size and with the number of concurrent clients?

RQ3
: Does the answer change when the Security Servers run on separate machines instead of one host, and how sensitive is it to added delay between them?

The contribution is a measurement rather than a new method: a testbed that can be redeployed on one or several hosts, the cost of each mechanism in milliseconds and bytes from three environments, a per-stage breakdown taken from X-Road's own operational monitoring, the sensitivity of the costs to added delay together with a prediction made before that measurement, and the raw data.

2 Background: what is paid per message

X-Road is a centrally managed distributed data exchange layer. A Central Server distributes the global configuration — the member registry, the trusted Certification Authorities (CAs) and the trusted time-stamping authorities — and each member operates a Security Server that handles key management, signing, secure transport, logging and access control [2, 11].

For this study the mechanisms divide by their position relative to the request path:

- **Synchronous**, paid on every message: the mutually authenticated TLS channel between the Security Servers; signing with the member's signing key; verification of the incoming signature; and writing request and response into the message log, whose bodies are

logged in full by default [12]. How much the TLS channel costs per message depends on whether connections are reused [13]. By default they are not: the client-side pool reuse (`pool-enable-connection-reuse`) and the provider-side support for pooled connections (`server-support-c` are both off, so each message opens a new TCP connection and performs a TLS handshake [12]. In our traffic captures this is a TLS 1.2 handshake resumed from a session ticket, without a certificate exchange, which costs one round trip on top of the TCP one.

- **Asynchronous**, amortised or deferred: certificate status checking, which uses cached OCSP responses refreshed on a fixed interval (20 minutes by default) rather than one query per message; time-stamping, applied in batches over a hash chain of log records (once per minute by default, with a tolerated time-stamping outage of four hours, the parameter `acceptable-timestamp-failure` being 14400 seconds); and the periodic refresh of the global configuration [9, 12, 1].

The split is a documented design decision: caching status responses and deferring time stamps were introduced to keep a per-message PKI operation from becoming both a single point of failure and a latency source [1]. It carries a security price as well, since a cached, nonce-less OCSP response leaves a window during which a revoked certificate is still accepted [2]. The performance side of that trade-off is what we measure.

Two further properties shape the experiment. First, messages are signed individually while time-stamping is batched over hash chains [11, 1], so the per-message cost of time-stamping depends on load and has to be observed across concurrency levels rather than at a single point. Second, the intervals above are defaults, and a production deployment may use longer ones: for the Estonian instance of 2017 a master's thesis documents a global configuration validity of six hours and an OCSP freshness of eight hours [8].

3 Methodology

3.1 Testbed

The testbed consists of one Central Server, a test CA providing certificate status and time-stamping services, four Security Servers (a management server, one service consumer and two service providers) and two information systems. Four Security Servers rather than the usual two give a genuinely distributed deployment in which the consumer's traffic is directed at two independent providers, so that queueing at a single node is not mistaken for protocol cost.

The provider-side information system is an echo service returning a response of configurable size with negligible processing time. This is deliberate: in the closest existing study database time dominated and the effect of interest was not separable from it [7]. The trade-off is that our figures overstate the *relative* share of the trust mechanisms compared with a realistic service, so one calibration point uses an echo service that delays its response by 50 ms.

All components run as containers pinned by image digest, from a single composition used in every environment, with identical CPU and memory limits per container, so that environments differ in topology and network rather than in how much hardware each component receives.

3.2 Environments

The original intention was simpler: develop and debug the stack on a laptop, and collect the actual data on a server. Working through the plan showed that the choice of environment is not a detail of execution but part of RQ3, and that there are four variants worth considering, with different costs and different lead times. They are complementary rather than alternatives, and they can be dropped from the bottom of the list if the scope turns out to be too large.

A. Single host (laptop). All components on one machine (64 GB RAM available; per-container limits of 2 cores and 4 GB are applied so that results remain comparable with the other environments). There is no network between the Security Servers. This reproduces the conditions of the published measurements [4] and makes our numbers directly comparable with them. *Pitfalls:* the Windows/WSL2 stack adds overhead of its own, and the memory given to WSL2 has to be fixed in configuration, otherwise it varies between runs; the load generator competes with the containers for cores at higher concurrency and must be constrained and checked not to be the bottleneck.

B. Two or three laptops on a local network. Consumer and provider Security Servers on physically separate machines, the supporting components on a third machine or alongside the consumer. Requirements: machines of broadly comparable class, a wired switch rather than Wi-Fi, Docker on each, and about 50 GB of free disk on the machines hosting a Security Server. *Pitfalls:* hardware is heterogeneous, so part of any asymmetry between nodes is explained by the machines rather than by the network; measurements are hard for anyone else to reproduce; the network delay obtained here (a fraction of a millisecond) is of the same order as inside a data centre, so this variant adds a real network but not a different class of latency. Its main value is availability — it needs no approvals and can be used immediately to debug the stack while access to the cloud is being arranged.

C. Cloud virtual machines (University of Tartu High Performance Computing centre). The primary environment for the reported results, because the hardware is uniform, described, and free of foreground activity, which makes the runs reproducible by others.

- *C1, two or three virtual machines.* Consumer and providers on different machines, so the measured link exists. Indicative sizing: a core machine (Central Server, test CA, management Security Server) with 4 vCPU, 8 GB RAM and 40 GB disk; a consumer machine with 4 vCPU, 8 GB and 60 GB; a provider machine with 6 vCPU, 12 GB and 60 GB. Software: Ubuntu 24.04 LTS and Docker; TCP 5500, 5577, 80, 443 and 4001 reachable between the machines; the default Docker bridge network moved off 172.17.0.0/16, as the operator's documentation requires.
- *C2, a single virtual machine (fallback).* Everything on one instance of 8 vCPU, 32 GB RAM and 60 GB disk, no externally opened ports. There is no network between the Security Servers, so RQ3 is addressed only through emulated latency (`tc netem` [16], 1–50 ms), as in TLS benchmarking where emulation isolates the effect of latency and does not need Internet-scale infrastructure [15]. It is a weaker answer than a real wide-area link and is reported as such.

Pitfalls: lead time for the virtual machines and for opening the ports is outside our control, which is why the request is sent on day one and why variant B exists.

D. Hybrid: consumer on a laptop, the rest in the cloud (optional). The only variant with a wide-area link and therefore with a round-trip time an order of magnitude above the others. It requires no inbound access to the laptop, since the consumer always initiates the connection; it does require TCP 5500 and 5577 to be reachable on the cloud side, with a WireGuard tunnel as a fallback if that is not granted — in which case a control run with and without the tunnel is needed to separate the tunnel's own cost. *Pitfalls:* a home uplink is variable, and the variance may well be comparable to the effects we measure, as happened in [7]. This variant is therefore treated as an external-validity check rather than a source of primary results.

If the scope has to be reduced, D is dropped first and B second; A together with C is the minimal meaningful set.

3.3 Isolating the cost of a mechanism

Signing cannot be switched off in X-Road, so the decomposition is obtained by differencing a ladder of configurations:

1. **Direct HTTP** from the client to the echo service, without X-Road. Lower bound.
2. **Plain mutual TLS** between two simple proxies using the same certificates. Separates the cost of the channel from the cost of X-Road.
3. **X-Road, default configuration.** The reference point for normal operation.
4. **X-Road without time-stamping**, with `acceptable-timestamp-failure-period` set to zero and the time-stamping service disabled, a configuration in which the Security Server has been observed to keep processing requests independently of the TSA [8].
5. **X-Road with immediate time-stamping** (`timestamp-immediately`), which removes batching and exposes the full per-message cost.
6. **X-Road with a short OCSP freshness interval**, so that status checking is forced into the request path.
7. **X-Road with elliptic-curve keys** (secp256r1) instead of the default RSA-2048.

Differences (3)−(2) and (2)−(1) give the cost of X-Road above plain mutual TLS and the cost of the channel itself; (5)−(3), (6)−(3) and (3)−(4) give time-stamping and status checking explicitly. Configurations 4 to 6 are deliberately set away from their defaults in order to make a mechanism observable, and the figures obtained from them describe an upper bound on its cost; configuration 3 is what a deployment actually runs, and production deployments may use even longer intervals [8].

In parallel, X-Road's own operational monitoring records per-request timestamps and message sizes inside the Security Server, which we intend to use as an independent, non-differential decomposition. The exact fields and the query interface are to be confirmed against the protocol specification before the first run.

3.4 How deep to go into signature schemes

A deliberate scoping decision, and one where the supervisor's opinion is wanted. We do not intend to study the internals of signature schemes, prove anything about them, or implement one. A signature is treated as a component with a measurable cost profile: time to sign, time to verify, bytes added to the message, behaviour under concurrency.

Inside X-Road two schemes can be exercised today, since both are supported for signing and authentication keys: RSA-2048 and the Elliptic Curve Digital Signature Algorithm (ECDSA) over secp256r1 [5, 4]. Comparing them costs one configuration change and yields a second data point for the signing and verification share, which is what makes any extrapolation credible: a single measured scheme gives a number but not a slope.

Post-quantum schemes cannot be exercised inside X-Road at present; no implementation exists and both available reports treat the migration as future work [5, 6]. We therefore propose two layers. The system layer measures RSA-2048 and ECDSA in the real exchange. The primitive layer measures signing and verification of the Module-Lattice Digital Signature Algorithm (ML-DSA, formerly Dilithium) and Falcon — together with the corresponding RSA and ECDSA operations for calibration — on the same hardware, using an off-the-shelf library, so that the ratios we use are ours rather than borrowed from a benchmark on different hardware. The prediction for a post-quantum X-Road is then the measured per-message signing share scaled by the measured primitive ratio, plus the measured effect of message size on latency applied to the

known growth in signature size (4595 bytes for Dilithium5 against 256 bytes for RSA-2048 [6]). Such a prediction is an extrapolation and will be labelled as one; its value is that each factor in it is measured rather than assumed. If a narrower scope is preferred, the primitive layer is the part to drop: the study remains sound with RSA and ECDSA alone and loses only the forward-looking section.

3.5 Factors, metrics and procedure

Payload sizes of 1 KB, 10 KB, 100 KB and 1 MB; concurrency of 1, 2, 4, 8, 16 and 32 clients; the seven configurations above; two key types; Representational State Transfer (REST) as the primary protocol. The grid is not fully crossed: sizes are crossed with configurations at a single client to obtain clean latency figures, and concurrency is crossed with configurations at 10 KB to obtain behaviour under load.

Metrics: median, 90th and 99th percentile latency (means are reported only for comparability with prior work); throughput at saturation; bytes on the wire per exchange; the number of status and time-stamping requests per thousand messages; message-log growth per message; and container-level CPU and memory use.

Each configuration is preceded by 50 warm-up requests and measured over 1000 requests, following [4]. Warm-up, independent repetitions and reporting the spread between them follow the practice for Java performance evaluation in [14], since the Security Server runs on the Java virtual machine. Measurements are repeated three times at different times of day, with the order of configurations randomised rather than run in blocks. At 1 MB payloads the count is reduced to 200 requests, which is sufficient at that size and keeps the message log from growing without need. Before the campaign a control run measures the same configuration twice in succession; if the difference between repetitions is comparable to the effect we expect, the testbed is not ready and the load or the node count is reduced. Raw per-request data, image digests and configuration files are published alongside the aggregates.

Storage deserves a note, because message bodies are logged in full by default on both Security Servers [12]. One exchange costs roughly twice the payload plus a few kilobytes of signature and metadata per server, so a thousand exchanges at 10 KB produce on the order of 60 MB in total and at 1 MB several gigabytes. The message log is therefore truncated between configurations and `keep-records-for` is set to one day, which keeps the peak requirement at a few gigabytes rather than letting it accumulate across the campaign.

3.6 Threats to validity

The test CA, status responder and TSA run next to the Security Servers, whereas in production they are external services with their own latency; our figures for those two mechanisms are therefore optimistic. The echo service raises the relative share of the trust mechanisms, which the 50 ms calibration point mitigates but does not remove. The weakened configurations are upper bounds rather than descriptions of normal operation. Containerisation, and on the laptop the Windows/WSL2 stack, add overhead of their own, which is one of the reasons for running environments B and C at all.

4 Expected outcome

A table mapping each trust mechanism to milliseconds and bytes per message, obtained in several environments, supported by raw data and a testbed that can be redeployed from a single composition file. Beyond the seminar report, the result is a reference point for three lines of work that currently lack one: post-quantum migration [5, 6], decentralised PKI [2] and distributed key management [4], each of which proposes to make trust more robust at a cost that has not been compared with the cost of what X-Road does today.

5 Plan

1. Request virtual machines and ports from the university computing centre (day one; the rest of the work proceeds in parallel).
2. Composition file and testbed, identical across environments; automation of member and service registration.
3. Echo service, load scripts, metric collection, message-log cleanup, and the control run for stability.
4. Environment A, full grid.
5. Environment C (or B, whichever becomes available first), same grid, plus emulated latency.
6. Environment D, reduced grid, if it is kept in scope.
7. Analysis, figures, report.

Steps 2 and 3 are the schedule risk; the waiting time in step 1 is the reason environment B is on the list at all.

References

[1] A. Ansper, A. Buldas, M. Freudenthal, J. Willemson. High-Performance Qualified Digital Signatures for X-Road. In *NordSec 2013*, LNCS 8208, pp. 123–138, Springer, 2013.

[2] M. Bakhtina, K. L. Leung, R. Matulevičius, A. Awad, P. Švenda. A Decentralised Public Key Infrastructure for X-Road. In *ARES 2023*, ACM. DOI: 10.1145/3600160.3605092.

[3] M. Bakhtina, R. Matulevičius, A. Awad, P. Kivimäki. On the Shift to Decentralised Identity Management in Distributed Data Exchange Systems. In *SAC '23*, pp. 864–873, ACM, 2023. DOI: 10.1145/3555776.3577678.

[4] M. Bakhtina, J. Kvapil, P. Švenda, R. Matulevičius. Securing Organisational Digital Identity: A Full-Lifecycle Approach to Distributed Key Management. *International Journal of Information Security*, 25:131, 2026. DOI: 10.1007/s10207-026-01291-5.

[5] M. Bakhtina. Migration of X-Road to Post-Quantum Cryptography. NIIS Publication series, Nordic Institute for Interoperability Solutions, 2026. ISBN 978-9916-9853-4-2.

[6] N. Snetkov, J. Vakarjuk. Integrating Post-Quantum Cryptography to UXP. Cybernetica Research Report D-2-499, 2023.

[7] E. Taklai. X-tee teenuste jõudlustestimine Eesti äriregistri näitel (Performance testing of X-Road services on the example of the Estonian Business Register). Diploma thesis, Tallinn University of Technology, 2020.

[8] L. Mironova. X-tee platvormi kriitiliste kesksete teenuste testimine (Testing of the critical central services of the X-Road platform). Master's thesis, Tallinn University of Technology, 2017.

[9] P. Kivimäki. Resisting Failure. NIIS blog, 20 November 2020. `https://www.niis.org/blog/2020/11/20/resisting-failure`.

[10] Estonian Information System Authority (RIA). X-tee Factsheet, Estonia. `https://www.x-tee.ee/factsheets/EE/`. Figures based on data of 1 September 2026; accessed 17 September 2026.

[11] Nordic Institute for Interoperability Solutions. X-Road: Security Server Architecture. `https://docs.x-road.global/Architecture/arc-ss_x-road_security_server_architecture.html`. Accessed 17 September 2026.

[12] Nordic Institute for Interoperability Solutions. X-Road: System Parameters (UG-SYSPAR). `https://docs.x-road.global/Manuals/ug-syspar_x-road_v6_system_parameters.html`. Accessed 17 September 2026.

[13] D. Naylor, A. Finamore, I. Leontiadis, Y. Grunenberger, M. Mellia, M. Munafò, K. Papagiannaki, P. Steenkiste. The Cost of the "S" in HTTPS. In *CoNEXT '14*, pp. 133–140, ACM, 2014. DOI: 10.1145/2674005.2674991.

[14] A. Georges, D. Buytaert, L. Eeckhout. Statistically Rigorous Java Performance Evaluation. In *OOPSLA '07*, pp. 57–76, ACM, 2007. DOI: 10.1145/1297027.1297033.

[15] C. Paquin, D. Stebila, G. Tamvada. Benchmarking Post-quantum Cryptography in TLS. In *PQCrypto 2020*, LNCS vol. 12100, pp. 72–91, Springer, 2020. Cryptology ePrint Archive 2019/1447.

[16] S. Hemminger. Network Emulation with NetEm. In *linux.conf.au 2005*, Canberra, 2005.

[17] R. C. Mayer, J. H. Davis, F. D. Schoorman. An Integrative Model of Organizational Trust. *Academy of Management Review*, 20(3):709–734, 1995.

[18] T. Grandison, M. Sloman. A Survey of Trust in Internet Applications. *IEEE Communications Surveys & Tutorials*, 3(4):2–16, 2000.

[19] R. Anderson. *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed. Wiley, 2020.